FOR COMPLETE BEGINNERS

Stop searching your list.

Complete Modern Java, episode 22: HashMap, in Java and TypeScript

> Link each name to its product, find it with one call, and handle a name that is not there.

A list keeps items in positions; finding Milk by name meant checking each one.

A map links each key to a value: get with the key gives back the value, without a search loop.

In this episode

- HashMap<String, Product>
- put and get
- One value per key
- remove
- A missing key: null
- containsKey
- TypeScript Map

Coming next

- Looping through a map (episode 23)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each run printed

put three; size(); get("Milk")	`3, 99`
put("Milk", 120) again; size()	`120, 3`
remove("Eggs"); size()	`2`
containsKey("Rice"); get("Rice")	`false, null`
rice.getPriceCents()	`NullPointerException`
catalog.ts: size, Milk, has Rice, get Rice	`3, 99, false, undefined`

Every row was run, not predicted.

02 Key and value

The first type is the key (a name), the second the value (a product). put links them; get finds the value by its key.

Putting the same key again replaces the value: the key now points to a different product. A HashMap does not promise insertion order.

Catalog.java

```
HashMap<String, Product> catalog = new HashMap<>();
catalog.put("Bread", new Product("Bread", 150));
catalog.put("Milk", new Product("Milk", 99));
catalog.put("Eggs", new Product("Eggs", 250));
IO.println(catalog.size());                    // 3
Product milk = catalog.get("Milk");
IO.println(milk.getPriceCents());              // 99
catalog.put("Milk", new Product("Milk", 120));
IO.println(catalog.get("Milk").getPriceCents());   // 120
IO.println(catalog.size());                    // 3
```

> Printed 3, 99, 120, 3; after remove("Eggs"), 2.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

03 A name that is not there

get returns null for a missing key, and asking null for its price throws. Check with containsKey first.

Safe.java

```
if (catalog.containsKey("Rice")) {
    Product rice = catalog.get("Rice");
    IO.println(rice.getPriceCents());
} else {
    IO.println("Rice is not in the catalog");
}
```

> Without the check: null, then NullPointerException. With it: Rice is not in the catalog.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

04 TypeScript Map

put is set, containsKey is has, remove is delete, size is a property, and a missing key gives undefined.

catalog.ts

```
const catalog = new Map<string, Product>();
catalog.set("Milk", new Product("Milk", 99));
const milk = catalog.get("Milk");
if (milk !== undefined) {
    console.log(milk.priceCents);   // 99
}
console.log(catalog.get("Rice"));   // undefined
```

> tsc was quiet; node printed 3, 99, false, undefined.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

Use a map

1. Look things up by a name? A map.
2. put links a key to a value; get finds it.
3. One value per key: put again replaces.
4. Missing key: null (Java) or undefined (TypeScript). Check first.
5. Challenge: Coffee at 350, then Tea without crashing.

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-11-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.